Lecture 03.14: Functions in Python (Syntax, Scope & Call Stack)

📌 Executive Summary

Function definition, parameter passing, return statements, local vs global scope (LEGB rule), and CPython call stack activation frames.

🔬 Theoretical Foundations & Principles

1. The Mathematical & Systems Abstraction of Functions

A function is an encapsulated, reusable computation mapping domain arguments to a codomain output:

$$f : \mathcal{A} \to \mathcal{B}$$

2. Call Stack & Activation Records (`PyFrameObject`)

When a function is invoked in Python:

1. CPython creates a `PyFrameObject` containing local variable arrays, value stack, and instruction pointer.
2. The frame is pushed onto the thread's call stack.
3. Upon reaching `return`, the return value is pushed onto the caller frame's value stack, and the frame is popped and deallocated.

3. The LEGB Scope Resolution Rule

Python resolves variable identifiers using the strict LEGB order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

💻 Production Implementation & Code

```
def calculate_circle_metrics(radius: float) -> tuple[float, float]:
    """Calculates area and circumference of circle."""
    import math
    area = math.pi * (radius ** 2)
    circumference = 2 * math.pi * radius
    return area, circumference

a, c = calculate_circle_metrics(5.0)
print(f"Radius 5.0 -> Area: {a:.2f}, Circumference: {c:.2f}")
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.